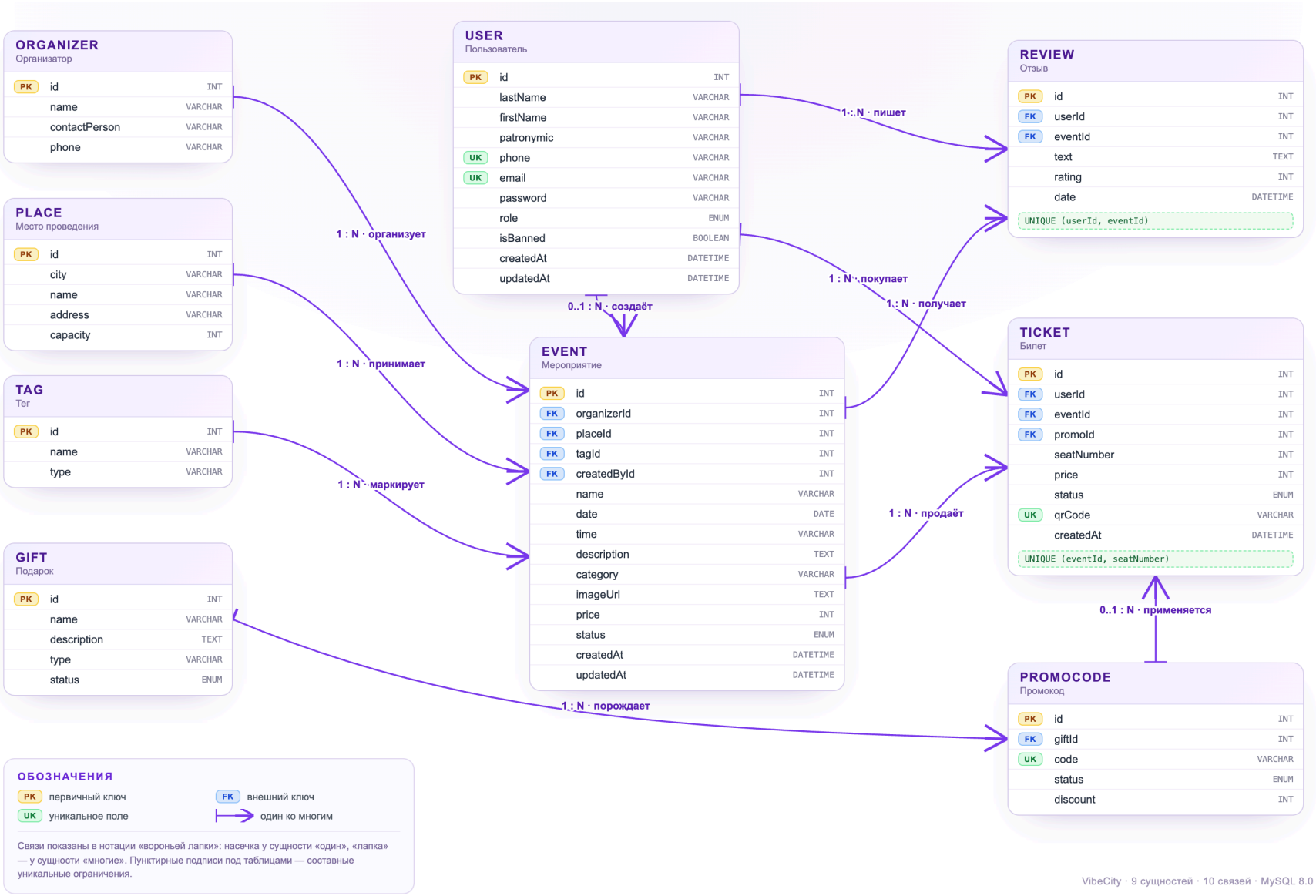
КУРСОВОЙ ПРОЕКТ · VIBECITY

Проектирование базы данных

Афиша мероприятий · 9 сущностей · 10 связей · MySQL 8.0

Схема: backend/prisma/schema.prisma



PK — первичный ключ · FK — внешний ключ · UK — уникальное поле · Связи — в нотации «вороньей лапки»

Интерактивная версия: docs/database/er-diagram.html

КУРСОВОЙ ПРОЕКТ · VIBECITY

Проектирование базы данных

Афиша мероприятий: 9 сущностей, 10 связей, MySQL 8.0, Prisma ORM.

Схема: backend/prisma/schema.prisma · Диаграмма: docs/database/er-diagram.html · Сценарий показа: docs/database/demo-script.md

1. Состав схемы

Сущность	Назначение	Ключевые поля и связи
`User`	Пользователи и администраторы	`email`, `phone` — UK; `role`; `isBanned`; 1:N отзывы, билеты, созданные события
`Organizer`	Справочник организаторов	`name`, `contactPerson`, `phone`; 1:N мероприятия
`Place`	Площадки проведения	`city`, `name`, `address`, `capacity`; 1:N мероприятия
`Tag`	Теги для навигации	`name`, `type`; 1:N мероприятия
`Event`	Мероприятие — центральная сущность	FK `organizerId`, `placeId`, `tagId`, `createdById`; `date`, `time`, `price`, `status`; 1:N билеты и отзывы
`Review`	Отзыв зрителя	FK `userId`, `eventId`; `rating` 1–5; UNIQUE(`userId`, `eventId`)
`Gift`	Подарок / бонус	`name`, `description`, `type`, `status`; 1:N промокоды
`Promocode`	Одноразовый промокод	FK `giftId`; `code` — UK; `status`, `discount`; 1:N билеты
`Ticket`	Купленный билет	FK `userId`, `eventId`, `promoId`; `seatNumber`, `price`, `qrCode` — UK; UNIQUE(`eventId`, `seatNumber`)

2. Связи и правила целостности

Связь	Кардинальность	Внешний ключ	ON DELETE
`Organizer` → `Event`	1 : N	`Event.organizerId`	`RESTRICT` — нельзя удалить организатора с событиями
`Place` → `Event`	1 : N	`Event.placeId`	`RESTRICT` — нельзя удалить площадку с событиями
`Tag` → `Event`	1 : N	`Event.tagId`	`RESTRICT`
`User` → `Event` (автор)	0..1 : N	`Event.createdById`	`SET NULL` — событие остаётся
`User` → `Review`	1 : N	`Review.userId`	`CASCADE`
`Event` → `Review`	1 : N	`Review.eventId`	`CASCADE`
`User` → `Ticket`	1 : N	`Ticket.userId`	`CASCADE`
`Event` → `Ticket`	1 : N	`Ticket.eventId`	`CASCADE`
`Promocode` → `Ticket`	0..1 : N	`Ticket.promoId`	`SET NULL` — билет сохраняется
`Gift` → `Promocode`	1 : N	`Promocode.giftId`	`CASCADE`

3. Ключи, уникальность и индексы

Первичные ключи

- во всех 9 таблицах — автоинкрементный `id`;
- суррогатный ключ не зависит от бизнес-данных.

Уникальные поля

- `User.email`, `User.phone`;
- `Promocode.code`;
- `Ticket.qrCode` (UUID).

Составные ограничения

- `Review(userId, eventId)` — один отзыв на событие;
- `Ticket(eventId, seatNumber)` — место не продаётся дважды.

Индексы

- `Event(date)`, `Event(category)`, `Event(status)`;
- `Review(eventId)`, `Ticket(userId)`.

ENUM: `User.role` USER / ADMIN · `Event.status` PENDING / APPROVED / REJECTED · `Ticket.status` ACTIVE / USED / CANCELLED · `Promocode.status` ACTIVE / USED / EXPIRED · `Gift.status` AVAILABLE / RESERVED / GIVEN.

4. Нормализация: 3НФ

1НФ. Все атрибуты атомарны: нет массивов, вложенного JSON и повторяющихся групп; дата и время хранятся отдельными полями.

2НФ. Используются суррогатные первичные ключи; неключевые атрибуты зависят от всего ключа, частичных зависимостей нет.

3НФ. Устранены транзитивные зависимости: город и адрес — в `Place`, контакты — в `Organizer`, данные подарка — в `Gift`, автор отзыва — в `User`. Рейтинг считается как `AVG(Review.rating)`, свободные места — как `Place.capacity - COUNT(Ticket)`, а не хранятся в `Event`.

Осознанные исключения: `Ticket.price` — исторический снимок цены на момент покупки; `Event.status` — состояние модерации; `Event.category` — строковый атрибут по ТЗ.

5. Решения, которые стоит объяснить

Решение	Зачем
`Event.price` + `Ticket.price`	Текущая цена события и зафиксированная цена уже купленного билета — разные вещи.
`UNIQUE(eventId, seatNumber)`	Защита от двойной продажи места даже при одновременных запросах.
`Event.status`	Модерация: пользовательские события уходят в PENDING, публикуются только APPROVED.
`Event.createdById`	Кабинет «мои мероприятия» и ограничение редактирования автором.
`User.isBanned`	Блокировка без удаления истории билетов и отзывов.
Транзакция при покупке	Создание билета и списание промокода выполняются атомарно.

6. Быстрая проверка в MySQL

```
docker compose exec db mysql --default-character-set=utf8mb4 \
  -uvibecity -pvibecity vibecity -e "SHOW CREATE TABLE Ticket\G"

docker compose exec db mysql --default-character-set=utf8mb4 \
  -uvibecity -pvibecity vibecity -e "SELECT e.name, p.city, o.name, t.name
  FROM Event e
  JOIN Place p ON p.id = e.placeId
  JOIN Organizer o ON o.id = e.organizerId
  JOIN Tag t ON t.id = e.tagId LIMIT 5;"
```

Полный сценарий показа на 7–8 минут, вероятные вопросы и SQL-примеры — в docs/database/demo-script.md.